# Prueba Técnica – Practicante de Desarrollo SW

| Campo | Valor |
|---|---|
| Apellidos y nombres | |
| Fecha de entrega | |
| Repositorio | |

## Contexto

El área de Soporte TI necesita una herramienta web para registrar, asignar y hacer seguimiento a los tickets de soporte de los usuarios internos. El auxiliar de TI registra cada solicitud, la asigna a un técnico según su disponibilidad y carga de trabajo, y la lleva hasta su cierre. La solución debe ser una sola página (SPA): toda la interacción ocurre por AJAX, sin recargar la página.

## Requerimientos funcionales

### 1. Registro de ticket

| Campo | Regla |
|---|---|
| Número de ticket | Autogenerado, único, formato `TK-AAAA-NNNNNN` (ej. `TK-2026-000042`). El consecutivo reinicia cada año y no puede repetirse aunque dos usuarios registren al mismo tiempo. |
| Código del usuario solicitante | Al escribirlo, el sistema busca por AJAX y carga nombre, cargo y área. Si el código no existe o el usuario está inactivo, no se permite registrar. |
| Categoría | Hardware / Software / Red / Otro (desde tabla en BD, no quemada en el HTML). |
| Descripción | Obligatoria, mínimo 20 caracteres, máximo 1.000. |
| Prioridad | Alta / Media / Baja. |
| Técnico asignado | Solo técnicos activos. La lista muestra los tickets abiertos o en curso de cada técnico y sugiere por |

| Campo | Regla |
|---|---|
| | defecto el de menor carga. Un técnico con 5 tickets activos no puede recibir más. |

## 2. Flujo de estados

Un ticket solo puede avanzar ABIERTO > EN CURSO > CERRADO. No se puede saltar de ABIERTO a CERRADO ni reabrir un ticket cerrado. Para cerrar es obligatorio registrar la solución aplicada. Cada cambio (estado o reasignación de técnico) queda en un historial con fecha, hora, estado anterior, estado nuevo y observación.

## 3. Acuerdo de nivel de servicio (SLA)

| Prioridad | Tiempo máximo de solución |
|---|---|
| Alta | 4 horas |
| Media | 24 horas |
| Baja | 72 horas |

Cada ticket calcula su fecha límite al crearse. La tabla muestra un indicador de "Vencido" o "Por vencer" (queda menos del 25% del tiempo) en los tickets no cerrados.

## 4. Tabla de tickets

- Columnas: número, usuario, categoría, prioridad, técnico, estado, fecha de creación, indicador SLA.
- Se actualiza sin recargar al registrar, cambiar estado o reasignar.
- Filtros por estado, prioridad, técnico y texto libre (número o nombre de usuario).
- Paginación del lado del servidor, 10 registros por página.
- Al hacer clic en una fila se abre el detalle con el historial completo.

## 5. Tablero resumen

Encima de la tabla: total de tickets por estado, tickets vencidos y carga activa por técnico. Se refresca junto con la tabla.

## API REST

La API vive en el mismo proyecto PHP, responde siempre JSON (Content-Type: application/json) y exige el encabezado X-API-Key con una clave definida en un archivo de configuración. Sin clave o con clave inválida responde 401. La propia interfaz web debe consumir esta API, no consultas sueltas por archivo.

| Método | Ruta | Descripción | Respuestas esperadas |
|---|---|---|---|
| GET | `/api/tickets` | Lista paginada. Filtros por query string: `estado`, `prioridad`, `tecnico`, `q`, `page`, `per_page` | 200 |
| GET | `/api/tickets/{numero}` | Detalle del ticket con su historial | 200, 404 |
| POST | `/api/tickets` | Crea un ticket | 201 con el ticket creado, 422 si falla validación |
| PATCH | `/api/tickets/{numero}/estado` | Cambia el estado; body `{"estado": "EN CURSO", "observacion": "..."}` | 200, 404, 409 si la transición no es válida, 422 |
| PATCH | `/api/tickets/{numero}/tecnico` | Reasigna técnico | 200, 404, 409 si el técnico está al tope de carga |
| GET | `/api/usuarios/{codigo}` | Datos del solicitante para la captura | 200, 404 |
| GET | `/api/tecnicos` | Técnicos activos con su carga actual | 200 |

El detalle y la lista incluyen como mínimo: número de ticket, usuario, categoría, prioridad, técnico asignado, estado, fecha de creación y fecha límite SLA.

Respuesta de lista:

```
{
  "data": [{
            "numero": "TK-2026-000042",
            "usuario": "...",
            "categoria": "Red",
            "prioridad": "Alta",
            "tecnico": "...",
             "estado": "EN CURSO",
             "creado": "2026-10-16T09:30:00-05:00",
             "vence": "2026-10-16T13:30:00-05:00"
           }],
  "meta": {
            "page": 1,
            "per_page": 10,
            "total": 37
            }
}
```

Formato único de error:

```
{
"error": {
          "code": "TRANSICION_INVALIDA",
          "message": "Un ticket ABIERTO no puede pasar a CERRADO",
          "fields": {}
          }
}
```

## Requisitos técnicos

- PHP 8.x sin framework de backend (se permite Composer para autoload o utilidades). JavaScript con fetch o jQuery AJAX; sin React/Vue/Angular. CSS libre (Bootstrap permitido).
- Base de datos de su preferencia (MySQL, MariaDB, PostgreSQL o SQL Server).
- Código separado por capas: rutas/controladores, lógica de negocio y acceso a datos. No se acepta SQL dentro del HTML ni un único archivo con todo.

- Consultas con sentencias preparadas (PDO). Salida escapada en el navegador para evitar XSS.
- Credenciales de BD y API key en un archivo de configuración fuera del código, con un ejemplo versionado (config.example.php o .env.example).
- Control de errores: ninguna respuesta de la API puede mostrar trazas de PHP o errores SQL al cliente.

## Base de datos

El script debe crear el esquema completo y cargar datos de prueba:

- Tablas mínimas: usuarios, tecnicos, categorias, tickets, ticket_historial.
- Llaves primarias, foráneas, restricciones UNIQUE y NOT NULL donde correspondan; índices para los filtros de la tabla.
- Datos semilla: al menos 10 usuarios (uno inactivo), 4 técnicos (uno inactivo), las 4 categorías y 25 tickets en distintos estados, incluidos algunos con SLA vencido.
- La generación del número de ticket debe ser segura ante concurrencia (transacción, bloqueo o tabla de consecutivos); explique en el video cómo lo resolvió.

## Entregables y plazo

Plazo: 3 días calendario desde la recepción de esta prueba.

1. Repositorio Git público (GitHub, GitLab o Azure DevOps) con historial de commits real; no se acepta un único commit con todo el proyecto. Se entrega además un ZIP del repositorio.
2. Script de base de datos con esquema y datos semilla.
3. README.md con requisitos, pasos de instalación, configuración y ejemplos de llamada a cada endpoint.
4. Colección de Postman (o archivo .http) con todas las peticiones de la API, incluidos casos de error.
5. Video de máximo 12 minutos: demostración completa del flujo (registro > en curso > cierre, filtros, SLA, errores de validación y transición inválida), recorrido por el código y explicación de dos decisiones técnicas que haya tomado.

Si usa herramientas de IA, indíquelo en el README. Está permitido, pero en la sustentación deberá explicar y modificar en vivo cualquier parte de su código.

## Puntos extra (opcionales)

- docker-compose.yml que levante la aplicación y la BD con un solo comando.
- Pruebas automatizadas (PHPUnit) de las reglas de negocio: transiciones de estado, tope de carga y cálculo de SLA.
- Exportar a CSV el listado filtrado.
- Despliegue accesible en línea.

## Criterios de evaluación

La prueba se califica sobre 100 puntos; los puntos extra suman hasta 10 adicionales. Se requieren 70 puntos para pasar a la sustentación.

| Criterio | Qué se revisa | Puntos |
|---|---|---|
| Funcionalidad | Registro, flujo de estados, SLA, filtros, paginación y tablero funcionan sin recargar la página | 30 |
| API REST | Endpoints completos, códigos HTTP correctos, formato de error único, API key | 20 |
| Reglas de negocio y validación | Transiciones inválidas bloqueadas, tope de carga, validación en servidor, consecutivo seguro | 15 |
| Base de datos | Modelo normalizado, llaves e índices, script reproducible con datos semilla | 10 |
| Calidad y seguridad del código | Separación por capas, sentencias preparadas, escape de salida, configuración fuera del código, nombres claros | 15 |

| Criterio | Qué se revisa | Puntos |
| --- | --- | --- |
| Documentación y entrega | README, colección de Postman, historial de Git, video claro | 10 |
| **Total** | | **100** |